

PROYECTO ACADÉMICO DE SUSTENTACIÓN

Tecnologías Web · Sistemas Distribuidos · Arquitectura de Software

PE

SISTEMA WEB DE CLIMA Y DATOS METEOROLÓGICOS DEL PERÚ

Dossier Técnico Integral y Guía Maestra de Exposición

React 19 + Vite

FastAPI (Python 3.11)

SQLite + SQLAlchemy

Open-Meteo API

Equipo de Desarrollo: Grupo 02

Cobertura: 25 Departamentos / 40+ Ciudades

Año: 2026

1. Ficha Técnica Oficial del Proyecto

Componente	Tecnología / Herramienta	Función y Rol en el Sistema
Frontend Core	React 19 + TypeScript + Vite 5	Interfaz de usuario reactiva, modular, fuertemente tipada y de carga instantánea.
Estilos y Diseño	Tailwind CSS 3.4 + Lucide React	Diseño moderno (*glassmorphism*, modo oscuro/claro, responsive) e iconografía meteorológica.
Visualización Gráfica	Recharts 3	Gráficos interactivos de temperatura, sensación térmica, lluvia y velocidad de viento.
Mapeo Geográfico	Leaflet 1.9	Mapa interactivo centrado en Perú con capas dinámicas de temperatura, lluvia y radiación UV.
Backend Framework	FastAPI (Python 3.11+)	Framework web asíncrono de alto rendimiento para exponer la API REST y procesar datos.
Servidor ASGI	Uvicorn 0.24	Servidor web asíncrono para despachar las peticiones del backend.
Validación de Datos	Pydantic 2.5 + Pydantic-Settings	Tipado estricto de entradas/salidas y gestión centralizada del archivo .env.
Cliente HTTP Backend	HTTPX 0.25 (Asíncrono)	Conexión no bloqueante a la API externa de Open-Meteo.
Base de Datos	SQLite con SQLAlchemy 2.0 ORM	BD relacional con 4 tablas para departamentos, ciudades, caché y favoritos.
Caché	In-Memory Cache (TTL 15m) + BD	Optimización para responder en < 50ms y prevenir sobrecarga de peticiones externas.
API Meteorológica	Open-Meteo (ECMWF 9km / GFS 13km)	Modelos meteorológicos globales con interpretación de códigos WMO en español.
Zona Horaria	America/Lima (UTC-5)	Sincronización exacta con la hora oficial de la República del Perú.

2. Arquitectura del Sistema y Flujo de Datos

Patrón Arquitectónico: Arquitectura en Capas Desacoplada (*Layered Architecture / Client-Server*). El Frontend y Backend se comunican exclusivamente mediante protocolo **HTTP REST** intercambiando cargas útiles en formato **JSON**.

1. Capa de Presentación (Frontend) Dashboard Hero: Clima actual y 6 métricas clave. Pronóstico: Carrusel 24 horas y 7 días. Mapa Leaflet: Visualización de los 25 departamentos. Comparador: Comparación de 2 a 4 ciudades en paralelo. Centro de Alertas: Alertas climáticas por severidad. Exportación: Generación y descarga directa en CSV.	2. Capa de Negocio y Datos (Backend) Routers REST: Endpoints modulares validados con Pydantic. WeatherService: Capa de servicio y normalizador en español. Motor de Alertas: Cálculo de umbrales críticos (UV ≥ 11, heladas ≤ 0°C). Caché en Memoria: Almacén de respuestas con TTL de 15 minutos. Base de Datos Relacional: SQLite con catálogo peruano.
--	---



3. Base de Datos: Estructura, Tablas y Relaciones

Tipo de Base de Datos: Relacional (SQL / RDBMS) administrada con **SQLAlchemy ORM**. Se conecta vía `sqlite:///./clima_peru.db` (archivo local) y es 100% compatible con **PostgreSQL** para despliegues empresariales.

Detalle de las 4 Tablas Relacionales:

Tabla	Descripción y Propósito	Campos Principales	Relaciones
departments	Almacena los 25 departamentos del Perú con datos geográficos y centroides.	id (PK), name, code, capital, latitude, longitude, region_natural (Costa/Sierra/Selva), description.	1 a N con cities (eliminación en cascada).
cities	Catálogo de más de 40 ciudades y provincias con altitud georreferenciada.	id (PK), department_id (FK), name, province, latitude, longitude, altitude (m.s.n.m.), is_capital, is_featured.	N a 1 con departments.
weather_cache	Caché persistente para almacenar respuestas JSON y garantizar resiliencia offline.	id (PK), cache_key (String indexado, ej. weather_-12.04_-77.04), data_json (Text), created_at (DateTime).	Tabla independiente de optimización.
favorite_cities	Almacena las ciudades marcadas como preferidas por el usuario.	id (PK), city_id (FK a cities.id), created_at (DateTime).	1 a 1 con cities.

4. Conexión de APIs y Gestión de Tiempo Real

A. Conexión Backend → Open-Meteo API

El backend invoca a Open-Meteo mediante **HTTPX Asíncrono**:

- Parámetros enviados:** Coordenadas (lat, lon), variables horarias/diarias y zona horaria `America/Lima`.
- Interpretación WMO:** Traduce códigos numéricos meteorológicos estándar a español comprensible (ej. `0` = Cielo despejado, `95` = Tormenta eléctrica).
- Resiliencia:** Si la API externa no responde, el sistema lee la última captura válida en `weather_cache`.

B. Conexión Frontend (React) → Backend (FastAPI)

El cliente web consume los endpoints con **Fetch API**:

- Ubicado centralizadamente en `frontend/src/services/api.ts`.
- Soporta URL base configurable mediante variable de entorno `VITE_API_URL`.
- Tipado completo con interfaces TypeScript (`types/weather.ts`) garantizando cero errores de tipo en tiempo de ejecución.

5. Catálogo Completo de Endpoints REST

Método	Endpoint	Descripción	Parámetros
GET	/api/cities	Lista ciudades peruanas con filtro por departamento o región.	query, department_id, region
GET	/api/departments	Lista los 25 departamentos y sus respectivas capitales.	-
GET	/api/weather/forecast	Principal: Clima actual, 24 horas y pronóstico de 7 días.	city_id, lat, lon

Método	Endpoint	Descripción	Parámetros
GET	/api/weather/overview	Resumen climático de los 25 departamentos para el mapa.	-
GET	/api/alerts	Alertas meteorológicas calculadas por umbrales físicos.	city_id (opcional)
GET	/api/compare	Comparativa simultánea de 2 a 4 ciudades peruanas.	city_ids=1,5,10
GET	/api/history	Series temporales históricas y tendencias calculadas.	city_id, variable, days
GET	/api/rankings	Top 5 nacional: más calurosas, frías, lluviosas y mayor UV.	-
GET	/api/export/csv	Descarga de reporte meteorológico en formato plano CSV.	city_id, days
GET/POST	/api/favorites	Gestión de ciudades favoritas del usuario.	city_id

6. Explicación Línea por Línea de Dependencias y Archivos

A. Archivo `requirements.txt` (Backend en Python):

```
1: fastapi>=0.104.0      # Framework web asíncrono para construir la API REST y routers.
2: uvicorn>=0.24.0       # Servidor web ASGI de alto rendimiento para ejecutar FastAPI.
3: httpx>=0.25.0         # Cliente HTTP asíncrono para consultar la API de Open-Meteo.
4: sqlalchemy>=2.0.0     # ORM para mapear objetos de Python a tablas en la base de datos.
5: pydantic>=2.5.0       # Validación estricta de esquemas de datos y tipado en tiempo de ejecución.
6: pydantic-settings>=2.0.0 # Lectura y tipado seguro de variables de entorno (.env).
7: pytest>=7.4.0         # Framework para pruebas unitarias automatizadas de endpoints.
```

B. ¿Qué hay en `node_modules`? (Frontend):

Definición concisa: Es el directorio generado por `npm install` que contiene el código fuente compilado de todas las librerías de JavaScript y TypeScript que necesita React para funcionar (React 19, Leaflet para mapas, Recharts para gráficos y Tailwind CSS).

7. Comandos de Instalación y Ejecución

◆ Terminal 1: Backend (FastAPI)

```
# 1. Ingresar a la carpeta backend
cd backend

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Iniciar servidor con recarga en vivo
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Swagger Docs: <http://127.0.0.1:8000/docs>

◆ Terminal 2: Frontend (React + Vite)

```
# 1. Ingresar a la carpeta frontend
cd frontend

# 2. Instalar módulos de node
npm install

# 3. Iniciar entorno de desarrollo
npm run dev
```

App Web: <http://localhost:5173>

8. Guía de Sustentación: Discurso, Demo y Preguntas del Jurado

Discurso de Apertura (30 Segundos):

"Buenas tardes, nuestro proyecto es el **Sistema Meteorológico del Perú**, desarrollado por el **Grupo 02**. Es una plataforma web integral para consultar el clima en tiempo real, pronósticos horarios y alertas en los **25 departamentos y más de 40 ciudades del Perú**, combinando modelos meteorológicos globales, mapas interactivos con Leaflet y gráficos analíticos con Recharts."

Preguntas Típicas del Jurado y Respuestas Maestras:

1. ¿Por qué usaron FastAPI y no Flask o Django?

"FastAPI es nativamente asíncrono (`async/await`), lo que permite manejar múltiples peticiones concurrentes a la API meteorológica con bajísima latencia. Además, genera automáticamente la documentación interactiva Swagger y valida tipos con Pydantic."

2. ¿Cómo aseguran que el sistema no falle si se cae internet o la API externa?

"Diseñamos un mecanismo de resiliencia de 2 niveles: primero revisa la memoria RAM (TTL 15 min); si la API de Open-Meteo no responde, el backend rescata la última captura meteorológica válida almacenada en la tabla `weather_cache` de SQLite."

3. **¿Cómo se conectan el Frontend y Backend si están en puertos distintos?**

"FastAPI implementa CORSMiddleware, permitiendo solicitudes HTTP REST seguras desde el origen de React (puerto 5173 o dominio de producción) sin bloqueos por políticas de origen cruzado."

4. **¿Cómo se calculan las alertas meteorológicas?**

"Mediante un motor de reglas propio (ALertEngine) que evalúa en tiempo real umbrales físicos del SENAMHI y la OMS: Radiación UV ≥ 11 (Extremo), temperaturas $\geq 35^{\circ}\text{C}$ (Ola de calor), heladas $\leq 0^{\circ}\text{C}$ y vientos ≥ 40 km/h."